The `energese` Package User Guide

1 Introduction

`energese` renders H.T. Odum's Energy Systems Language (ESL) diagrams from a description of the system rather than from drawing commands. You state what the components are and how energy moves between them; the package places them and draws the result.

A system is described by three things: *nodes* (the components), *edges* (the pathways between them) and *metadata* (diagram-wide settings). That description can be written as JSON or directly in TEX. Both are parsed into the same model and rendered by the same code, so the two are interchangeable and produce identical output.

2 Requirements

LuaLATEX (TEX Live 2023 or later), TikZ with the `positioning`, `arrows.meta`, `shapes.geometric`, `calc` and `backgrounds` libraries, and the `luacode`, `environ` and `fontspec` packages. The `dkjson` Lua library ships with the package.

Put `energese.sty`, `energese-core.lua`, `energese-shapes.tex` and `dkjson.lua` somewhere on the TEX search path — typically the directory holding your document.

3 Authoring a diagram

There are three ways in. The first two share the layout engine; the third bypasses it.

3.1 From a JSON file

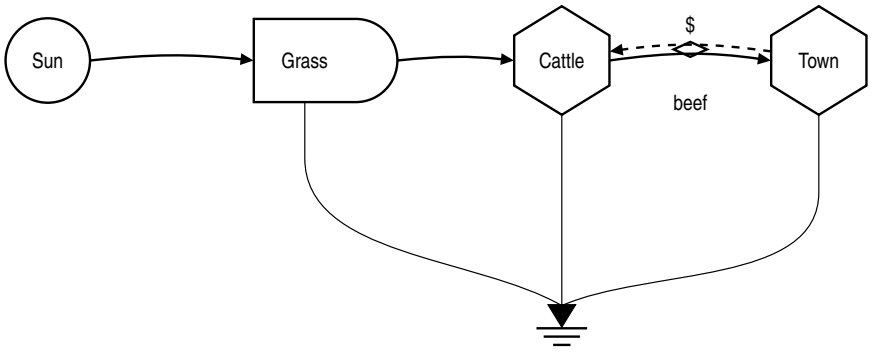
```
\renderEnergese{examples/simple_system.json}
```

An optional argument is passed straight through to the underlying `tikzpicture`:

```
\renderEnergese[scale=0.8]{examples/simple_system.json}
```

A `scale` shrinks the diagram as one picture, labels included: the layout is computed for symbols at their full size, so they must scale with the coordinates, and `transform shape` is added for you. A `xscale` or `yscale` is refused, because it would distort the symbols. To space a diagram out rather than shrink it, change `column_spacing` and `row_spacing`; to resize the symbols, `\energeseunit` (Section 11).

The path is resolved relative to the current directory first, then through `kpsewhich`, so a document is not tied to the directory it is compiled from. That call produces:



3.2 From inline JSON

For a one-off diagram that does not warrant its own file:

```
\begin{energeseData}
{
  "nodes": [
    {"id": "sun",   "type": "source",   "Tr": 1,   "label": "Sun"},
    {"id": "grass", "type": "producer", "Tr": 100, "label": "Grass"}
  ],
  "edges": [
    {"from": "sun", "to": "grass", "type": "energy"}
  ]
}
\end{energeseData}
```

3.3 From TEX

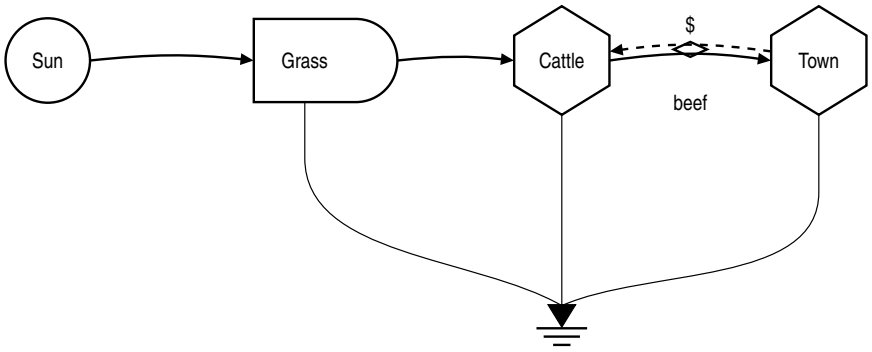
The `energese` environment describes the same model with declarative macros. Keys are exactly the JSON field names, so anything expressible in one front end is expressible in the other.

```
\begin{energese}
    \esmeta{column_spacing=3.4, row_spacing=1.5, show_heat_sink=true}

    \esnode{sun}{source}      {Tr=1,     label={Sun}}
    \esnode{grass}{producer}  {Tr=100,   label={Grass}}
    \esnode{cow}{consumer}    {Tr=2000,  label={Cattle}}
    \esnode{town}{consumer}   {Tr=10000, label={Town}}
    \esnode{market}{transaction}{seller=cow, buyer=town,
                                 energy_label={beef}, money_label={\$}}

    \esflow{sun}{grass}{energy}
    \esflow{grass}{cow}{energy}
\end{energese}
```

which renders as:



That is the same system as `examples/simple_system.json`, and it rasterises identically — the repository's test suite asserts as much.

Macro	Purpose
`\esnode{<id>}{<type>}{<keys>}`	Declare a node.
`\esflow{<from>}{<to>}{<type>}{<keys>}`	Declare an edge.
`\esmeta{<keys>}`	Set diagram metadata.
`\esboundary{<keys>}`	Add a system boundary.

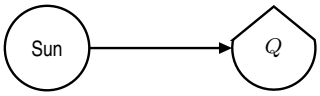
Table 1: The TEX authoring macros.

The trailing key list is optional on every macro. A value containing a comma must be braced — `label_options={midway, above}` — and a bare key is a flag, so `system_boundary` means `system_boundary=true`. Call `\esboundary` more than once for several boundaries.

3.4 Raw TikZ

The symbols and line styles are ordinary TikZ, so a diagram that wants no automatic layout can be drawn by hand:

```
\begin{tikzpicture}
    \node[energese source]  (sun) at (0,0) {Sun};
    \node[energese storage] (q)   at (3,0) {$Q$};
    \draw[energese energy] (sun.energy_out) -- (q.energy_in);
\end{tikzpicture}
```



4 The model

4.1 Nodes

Key	Required	Meaning
`id`	yes	Unique identifier, referenced by edges.
`type`	yes	One of the types in Section 11.
`Tr`	yes†	Transformity; sets the column.
`label`	yes†	Label text; LaTeX is permitted.
`short_label`	no	Odum's nomenclature for the node: `Q1`, `X2`.
`label_position`	no	`inside`, `above` or `below`.
`label_options`	no	TikZ options applied to the node.
`magnitude`	no	Scale factor for the symbol, not its label. Default `1.0`.
`x`, `y`	no	Manual coordinates, overriding the layout per axis.
`layer_hint`	no	`control` or `decomposition`: names the node's layer.
`y_gravity`	no	Nudges the node vertically, in diagram units.
`notched`	no	On a `consumer`: draw it with an interaction notch.
`inside`	no	Whether the system window encloses this node. Sources default to `false`, everything else to `true`.
`points_to`	no	On a `text` node: draw a dashed leader to that node.
`clearance_w`, `clearance_h`	no	Override the estimated extents used for routing.
`parent`, `parent_dx`, `parent_dy`, `attach`	no	On an `interaction` or `transaction`: the component it attaches to, the offset, or `attach=false` to keep it in a column of its own.
On an exchange (a `transaction` with a seller and a buyer)		
`seller`, `buyer`	yes	The two parties.
`energy_label`, `money_label`	no	Captions for the two tracks.
`energy_label_options`, `money_label_options`	no	Place a caption by hand.
`options`, `waypoints`	no	Route the goods; the money follows.

Table 2: Node keys. † not required for `text`, `box` or `ground` nodes, or for an exchange, which carry no transformity.

4.2 Edges

Key	Required	Meaning
`from`, `to`	yes	Node identifiers.
`type`	yes	Pathway type; see below.
`label`	no	The pathway's name (J_0).
`quantity`	no	Its quantity (2 g/day), written on the other side.
`label_options`	no	TikZ options for the label. Supplying this places the label by hand; without it the label is placed automatically.
`options`	no	TikZ options for the path, e.g. `bend left=20`.
`waypoints`	no	Explicit geometry; overrides automatic routing.
`from_anchor`, `to_anchor`	no	Override the chosen anchors.
`unused`	no	On a source's pathway: draw the unused-energy hook.
`heat`	no	Take the sender's dissipation off this pathway.
`volume`	no	Line width in points, to show relative flow.

Table 3: Edge keys.

Five pathway types are recognised. `energy` and `material` are forward flows and take part in the layout; `material` is exempt from the rule that quality rises along a pathway (Section 9). `information` is a solid

arrow arriving on the receiver's top. `feedback` is a higher-quality component serving a lower one, and arcs back over the top into the receiver's top, as Odum draws a city's services returning to the industry that built it. `money_feedback` is dashed and, unless the edge routes itself, arcs over the top too — money runs counter to energy.

Odum draws three line weights, and the package follows him: pathways between components at `\energeseflowwidth` (0.9 pt); heat-sink pathways at about half that, `\energeseheatwidth` (0.45 pt), so used energy reads as the fine tracery under the working system; and the system window at about one and a half, `\energeseframewidth` (1.35 pt). The ratios are measured from his figures. Set `volume` on an edge to widen it where the model wants to show relative flow.

Pathways are routed *around* any symbol that is not their own endpoint. In ESL a line meeting a symbol denotes flow into it, so a pathway crossing an unrelated symbol would assert a connection the model does not contain.

An edge that sets its own `bend` is checked too: the package keeps the direction of curvature you asked for and opens the angle until the pathway clears, falling back to automatic routing only if no angle does.

4.3 Exchanges

An exchange is one transaction with two coupled sides: goods pass one way, money the other, and the diamond is where they meet. Declaring the two as separate pathways would say something weaker — that a payment and a delivery both happened, not that they are the same transaction.

Give a `transaction` node a `seller` and a `buyer` and the package draws both sides:

```
\esnode{sale}{transaction}{seller=agri, buyer=consumer,
                           money_label={\$}, energy_label={food}}
```

The goods run from seller to buyer as one pathway, routed like any other. The money runs back beside them, about 1.2 mm to the left of the goods' direction of travel, following the same route, and both tracks pass through a small diamond centred between them — goods through one half, money through the other — as on Odum's symbol sheet. Each side's caption goes on its own outside. Track spacing and diamond size are measured from his figures.

The diamond is a *marker on the pathway*, not a component the pathway enters, so the lines cut straight through it, which every other symbol forbids. An exchange annotates a coupling and transforms nothing: it carries no transformity and no heat sink, and nothing may be anchored to it. A `transaction` used as an ordinary node — energy flowing into it and stopping — is not an exchange, and the energetic checks report it.

Goods that run down the hierarchy, from a higher transformity to a lower, are a service fed back rather than a forward flow, and are drawn and checked as `feedback`. `options` or `waypoints` on the exchange route its goods, and the money follows.

4.4 Geometry from the model

Automatic layout is a default, not an obligation. A modeller working in a drag-and-drop editor has already made every placement decision, and a renderer that re-derived them would publish something other than what was drawn. Geometry may therefore be supplied at both levels:

- `x` and `y` on a node override the layout engine, per axis and independently, so one component can be placed by hand while the rest are laid out automatically.
- `waypoints` on an edge override the router. In JSON they are a list of pairs, `"waypoints": [[3, 2], [5, 4]]`; in TEX the same written compactly, `waypoints={3,2; 5,4}`.

The two mix freely. A model carrying complete geometry renders exactly as it was drawn; one carrying none is laid out from scratch.

4.5 Metadata

`column_spacing` and `row_spacing` tune the layout grid; `label_mode` chooses which names appear (Section 5); `strict` turns the energetic checks into errors; `show_heat_sink` (default true) draws the dissipation pathways and their common ground; `heat_sink_label`, `heat_sink_y` and `ground_x` position and name it; `system_boundary` draws one or more enclosing rectangles, each taking `x_min`, `x_max`, `y_min`, `y_max`, and optionally `label` and `style`.

Setting `system_boundary` to `true` fits the frame automatically: a fixed margin outside the extents of everything the model puts inside the system, with room underneath for the dissipation funnel, which converges inside the frame so that a single line crosses it. Sources are outside by convention — a source is an input from beyond the system — and any node overrides that with `inside`. `boundary_margin` sets the margin in diagram units.

5 Labels

Odum names symbols as well as describing them: storages $Q_1, Q_2, \ldots$, sources and flows X and J. Both belong in the model — the short name is what the equations refer to, the long one is what a reader needs — so a node may carry `label` (the friendly name) and `short_label` (Odum's).

`label_mode` in the metadata chooses between them:

`label_mode`	Inside the symbol	Outside
`long` (default)	the friendly name	only if it will not fit
`short`	Odum's name	nothing
`both`	Odum's name	the friendly name

A symbol grows to hold its label, but only as far as its grid cell; a label larger than that would stop the symbol reading as the cell it occupies, so it is placed outside instead — below by default. That decision is automatic, and a node overrides it with `label_position`.

A displaced label is treated as part of its symbol for routing, so pathways go around the text as well as the glyph.

6 The conventions this package implements

Odum's Energy Systems Language has no written grammar. Its conventions are real but were never stated as rules, so any implementation has to decide what they are. This section records the decisions this package makes, so that a diagram it produces can be read — and disagreed with — precisely.

6.1 Horizontal position is ordinal in transformity

Distinct `Tr` values are sorted and assigned to consecutive columns at equal spacing. Transformities of 1, 100, 2000 and 10 000 become columns one to four.

Placement is therefore neither linear nor logarithmic in transformity. Linear placement is unusable, because transformity spans orders of magnitude and every low-quality component would collapse into the left margin. A logarithmic axis would be legible but would assert a metric the language never defines. Rank encodes exactly what the convention claims — the ordering — and nothing further.

Nodes sharing a transformity share a column, which makes a column an equivalence class of energy quality rather than a coordinate.

6.2 Vertical position is a role, not a quantity

Nothing in ESL assigns a magnitude to height, so the vertical axis carries no quantity of its own. In particular it does not carry power: along a transformation chain transformity rises because available power falls, so

power is already determined — inversely — by horizontal position, and encoding it vertically as well would be redundant or contradictory.

The vertical axis is not arbitrary either. It carries the same quality ordering as the horizontal, applied *relative to the main chain*, so the energy hierarchy runs from bottom-left to top-right.

Layer	Rule
On the axis	lies on the longest chain of *energy* pathways
Above	higher transformity than the chain node it connects to
Below	receives from the chain at equal or lower transformity
Fanned	anything the rules cannot classify, in declaration order

"Above" captures high-quality inputs, controllers and higher-quality products; "below" captures dissipation, decomposition and waste storage. This is why Odum draws a high-transformity fuel source entering a diagram from the top, and why the Sun — the largest power input to an ecosystem but the lowest in quality — sits at the bottom of the source column.

The chain is built from `energy` pathways only. A flow of degraded material to a waste store is a side branch, and counting it would let such a store capture the chain and sit on the axis when it belongs below it.

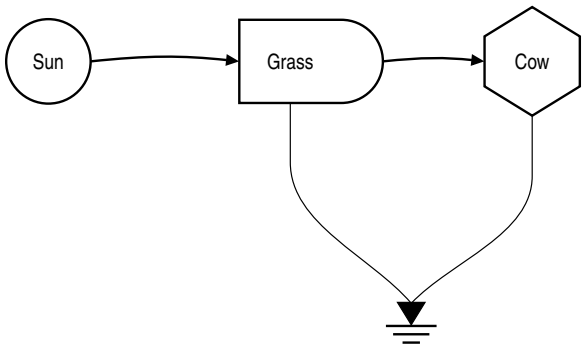
Set `layer_hint` to `control` or `decomposition` to name a layer explicitly; it wins over the derived one, including for a node lying on the main chain. `y_gravity` nudges a node, and an explicit `y` overrides everything.

Fanning uses declaration order, never node identifiers: a layout that changed when a node was renamed would not be reproducible.

6.3 The ground is not a component

If the horizontal axis orders components by energy quality, the heat sink at bottom centre looks like a violation — dissipated energy is the lowest quality present and ought to sit at the far left.

It is not a violation, because the ground is not a component. It carries no transformity, takes no part in the ordering, and terminates pathways rather than transforming anything:



The three components are ordered left to right by transformity. Dissipation runs downward — the vertical convention — and converges on a ground placed by neither rule. Three node types are annotations or terminators rather than components and are exempt from carrying transformity: `ground`, `text` and `box`.

6.4 Sources sit around the system

A source is an input from outside, and Odum places sources around the outside of the system in order of transformity: the lowest-quality on the left, the rest over the top, the most concentrated on the right, flowing in leftwards. The layout does the same. A source of higher transformity than every component goes on the right; one that is a second input to what it feeds, or of higher transformity than the lowest component, goes over the top of what it feeds; any other sits on the left, level with what it feeds. Sources on the top and right take no column of their own. An explicit `x` or `y` keeps a source where it is put.

6.5 A second input comes from above

The symbol sheet draws the interaction, the producer, the consumer and the box with their main input on the left and a second one arriving from above, and Odum's diagrams keep to it. Of a component's forward inflows, the one from the lowest-transformity sender enters on the left and each other enters at the top, unless its edge names an anchor.

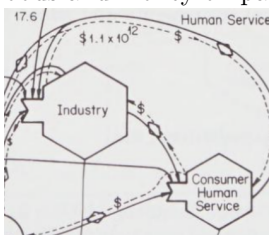
6.6 An interaction feeding a consumer is its notch

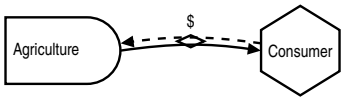
Where a consumer's inflows combine, Odum draws the interaction as a notch in the consumer's own left side rather than as a separate symbol. An unlabelled interaction whose forward flow goes to a single consumer is folded into it: the consumer is drawn notched and its inflows meet at the notch. A labelled interaction, one placed with `x`/`y`, or one with `attach=false` stays a symbol; `notched` on a consumer asks for the notch directly.

7 Following Odum's figures

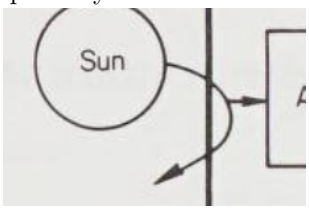
Each convention the package draws is taken from Odum's own figures, and `docs/odum-conventions.md` in the repository lists them with the measurements behind them. Here are the ones most easily got wrong, each with a crop of Odum's figure on the left and the package's drawing on the right. The same drawings are the repository's regression tests.

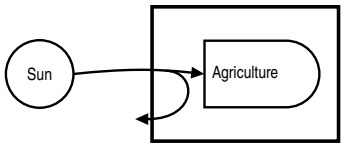
Exchange. Goods and money on paired, opposed tracks through one small diamond.



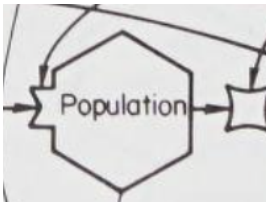


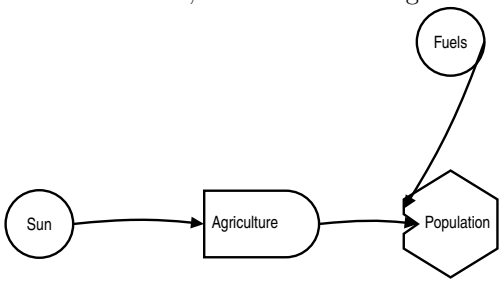
Unused energy. The part of a renewable inflow that reaches the system and leaves unused hooks back out where the pathway crosses the frame (`unused`).



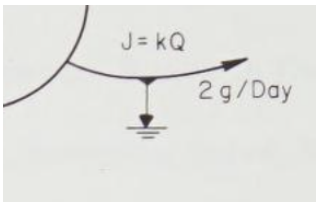


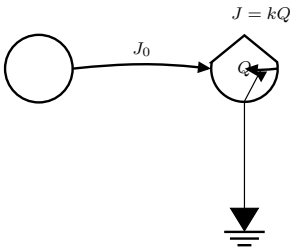
Notched consumer. The interaction drawn in the consumer's own side, its inflows meeting at the notch.



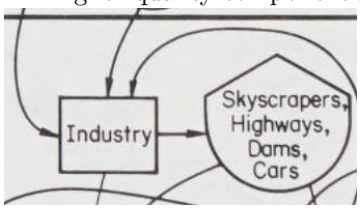


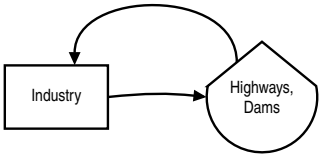
Heat taken off a pathway. A storage's depreciation as a branch off its outflow, marked at the junction (`heat`).



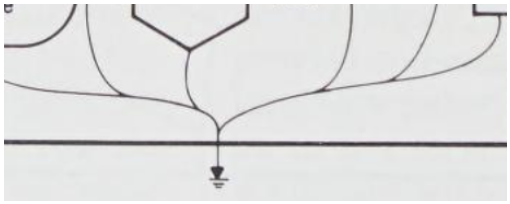


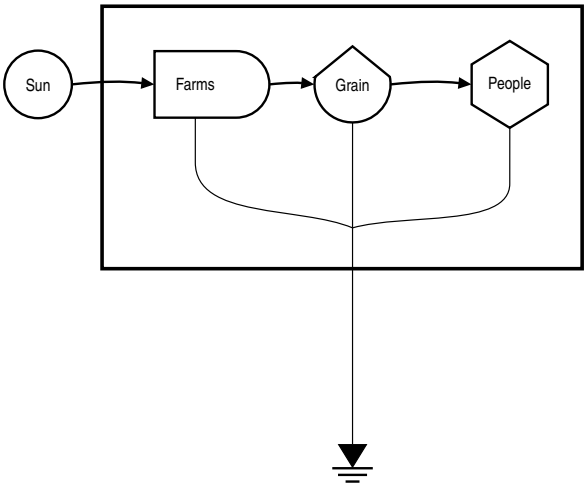
Feedback. A higher-quality component serving a lower one, arcing over into its top.





Used energy and the ground. Thin heat pathways merging inside the frame, one line leaving it, and one large arrowhead resting on the ground.





8 Reading a GSSK model

GSSK is the simulation kernel this notation is drawn for, and its model files are read directly:

```
\renderEnergese{model.json}
```

The same macro, because a GSSK file is normalised into an energese model before anything else looks at it. The point is that one file both simulates and draws, so a published figure comes from the model that produced the results beside it and the two cannot drift.

What the conversion does, beyond renaming fields:

- **Presentation is read out of `visual`.** GSSK's schema forbids extra keys on a node, and reserves `visual` for exactly this, so that is where `label`, `Tr`, `magnitude` and the rest belong. The file stays valid for the kernel.

- **Transformity is derived where the model has none**, as the length of the longest chain of pathways reaching each component. That orders the diagram; it is not an emergy calculation, and the values are $1, 2, 3$ rather than plausible-looking figures precisely so that nobody mistakes it for one. Supply `visual.Tr`, or a map under `metadata.energese.transformity`, to say otherwise.
- **Heat sinks are not carried across.** GSSK models dissipation as a `sink` node with pathways into it; this package knows a storage dissipates and draws the pathway and the ground itself. Both would draw it twice, so the sink and its pathways are dropped.
- **Canvas coordinates are converted**, from pixels with y downwards into diagram units with y upwards. The placement you made in an editor is kept; `metadata.energese.layout = auto` discards it and lets the transformity grid lay the diagram out instead, and `metadata.energese.pixels_per_unit` (default 50) sets the scale.

`docs/gssk-interop.md` in the repository has the full field mapping and the parts not yet implemented.

9 Energetic checks

Odum's stated reason for devising ESL was that scholars build models which are energetically invalid. A drawing tool cannot help, because it renders whatever is drawn. This package checks what a renderer can check without leaving the diagram:

1. Energy quality does not fall along an `energy` pathway.
2. Sources take no energy inflow; grounds emit none.
3. Every component is reachable from a source.
4. Energy that arrives somewhere leaves, as outflow or as dissipation.

Findings are reported as warnings, because a diagram that breaks a convention is often still worth drawing and the author is better placed to judge. Set `strict` in the metadata to make them errors instead.

9.1 What is deliberately not checked

`material` pathways are exempt from the first rule. Degraded material moving to a waste or decomposition store genuinely falls in transformity — that is what decomposition is — so applying the rule to material flows reports correct models as broken.

Conservation of energy is not checked. Balancing inflow against outflow and dissipation requires quantities, and a diagram carries none: `volume` sets a line width, not a flux. Conservation belongs with a simulation kernel working from the same model.

Emergy is not checked at all. The algorithms are not standardised and there is no formal definition to validate against, so a check would assert a standard that does not exist. Transformity positions a node; the package neither computes nor validates emergy.

The blunt summary: **if the model is energetically invalid, so is the diagram.** The package renders faithfully and does not repair.

10 Layout

Horizontal position comes from transformity: the distinct `Tr` values of the components are sorted and each is given a column, `column_spacing` apart (Section 6). Vertical position comes from the role a component plays relative to the main chain of `energy` pathways, rows `row_spacing` apart. Sources go around the outside, a component's second input comes from above, and an interaction feeding a consumer becomes its notch, all as described there.

Giving a node explicit `x` or `y` overrides the computed value for that axis alone, so a single stubborn component can be placed by hand without abandoning the layout for the rest. A figure that must follow an

original exactly — the worked example in Section 13 — is best traced: every position measured from the original, so its composition is the author's and not the layout engine's.

10.1 Modifiers travel with what they modify

An interaction is where two flows combine on their way into the thing they drive, and Odum draws it against that thing; a standalone transaction annotates a pair of parties in the same way. Given columns of their own by transformity, both drift away from the component they modify.

Interactions and transactions are therefore attached: their position is locked to a parent's, and they take no part in the column and row algorithms. The parent is the component they exchange energy with — a consumer for preference — and the offset follows the flow, upstream of the parent if the modifier feeds it and downstream if the parent feeds the modifier. Name a different parent with `parent`, change the offset with `parent_dx` and `parent_dy`, and switch the whole thing off for one node with `attach=false`. An explicit `x` or `y` also wins, as everywhere else.

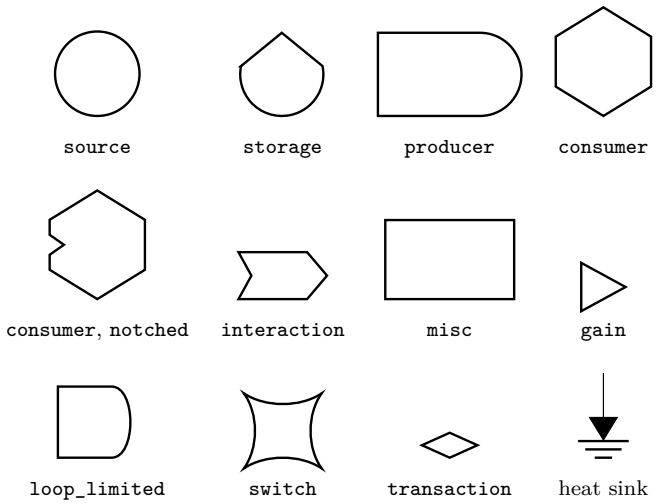
10.2 Pathways and their labels

A pathway that would cross a symbol which is not its own endpoint is routed around it: in this notation a line meeting a symbol means flow entering that symbol, so crossing one asserts a connection the model does not contain. The route is the shortest one that avoids every such symbol. An edge with its own `options` (a `bend`, say) keeps the author's curvature, and the angle is opened until it clears; `waypoints` hands the geometry over entirely.

A pathway's `label` is placed automatically, clear of the symbols, the other pathways, the dissipation bundle and the system frame so far as the diagram allows. Where symbols stand closer together than the label is wide there may be nowhere clear, and the least obstructed position is used. Supplying `label_options` places the label by hand instead.

11 Symbols, anchors and ports

The symbols, drawn bare as on Odum's sheet (`reference-images/complete-odum-symbols.png`), with the type that draws each:



Every symbol sizes itself to its label, growing its outline around the text so the label always sits inside. `minimum width` and `minimum height` set a floor for symbols with little or no text.

Each flow symbol carries the anchors `energy_in`, `energy_out`, `money_in`, `money_out`, `feedback_in` and `heat_sink`, in addition to `center`, `north`, `south`, `east` and `west`. `feedback_out` is defined on every symbol except `source` and `interaction`. Energy enters on the left and leaves on the right; a second input, a control

or a feedback arrives through `feedback_in` on top; money moves through the upper half; dissipation leaves through `heat_sink` at the bottom. On a notched consumer `energy_in` is the notch.

11.1 Ports

Every symbol carries a *connection ring*: points spaced a fixed distance apart along its outline, named by index as `ring0` at 0 degrees, counting anticlockwise. The spacing is `\energeseanchorpitch` (3 mm by default), so a larger symbol carries more points than a small one and a coupled pair attaching at adjacent indices is separated by the same distance whichever symbol it meets. The count is rounded to a multiple of four, which puts a point exactly on each axis and holds the spacing to within one point in four of the pitch.

These points are the symbol's *ports*, and they are the only places a pathway attaches. Two consequences follow, and they are what the renderer guarantees:

- **A named anchor is a port.** `energy_in` names a direction — where energy belongs, as Odum draws it — and the pathway attaches at the port nearest that direction. For the anchors on an axis that is the same point; elsewhere it is within half a pitch of it.
- **A port holds one pathway.** The second pathway to want a port takes the next one along, on the side it approaches from, so a fan of flows leaves a symbol as a fan rather than as one line drawn several times.

Neither is something to arrange by hand: `\renderEnergese` and the `energese` environment allocate ports for every pathway they draw. Hand-drawn TikZ can use the same machinery, and address a port directly where the choice of connection point is geometric rather than semantic:

```
\draw[energese energy] (a.ring3) -- (b.ring9);
\draw[energese money_feedback] (b.ring10) -- (a.ring2);
```

or ask for one and let the allocator place it:

```
\energeseresetports                  % begin a diagram
\energeseport{a}{energy_out}{b}      % a's port, facing b
\let\from\energeseportname
\energeseport{b}{energy_in}{a}
\draw[energese energy] (\from) -- (\energeseportname);
```

A symbol whose ports are all taken warns and doubles up rather than dropping the pathway. `box`, `text` and `ground` have no ring, and pathways meet them at the named anchor.

`gain` is the constant-gain amplifier, `loop_limited` the loop-limited converter, `switch` the on/off switching action and `misc` Odum's miscellaneous box, for a process with no symbol of its own. Because a triangle is a poor container for text, label a `gain` with a single character as Odum does; a long label there will overflow the apex.

Three further node types are plain TikZ rather than Odum symbols: `box` (a rectangle, for an equation or a note; it does not dissipate), `text` (an unboxed label, useful as a routing waypoint, and the node a `points_to` leader starts from) and `ground` (the heat-sink terminator glyph).

Symbol outlines are drawn at `\energeselinewidth`, which defaults to 0.9 pt to match the constant pen weight measured in Odum's published sheet. Redefine it, `\energeseflowwidth`, `\energeseheatwidth` or `\energeseframewidth` with `\setlength` for a heavier or lighter house style.

Every symbol holds its shape. A shape has a canonical aspect ratio measured from Odum's sheet, and it keeps that ratio at every size: a symbol grows uniformly to hold a longer label rather than stretching, because a widened hexagon reads as a different symbol. Where a label will not fit even so, the label is displaced rather than the shape distorted.

`magnitude` scales the symbol and not its label, so symbols of different magnitude carrying the same words are set in the same type size.

Sizes are set on an invisible grid. Each shape encloses its label differently, so sizing every symbol purely from its own text would make identical labels produce very different symbols. `\energeseunit` is one grid cell

(default 1.1 cm), and each symbol's minimum size is the multiple of a cell at which it lands on the relative height Odum draws it at. A label that will not fit its cell grows its own symbol. To scale a whole diagram's symbols together, set the unit:

```
\setlength{\energeseunit}{1.4cm}
```

Note that TikZ's `font` key is a single value, so a `label_options` of `font=\large` replaces the style's `font=\sffamily` rather than adding to it. Write `font=\sffamily\large` to keep the sans face.

12 Heat sinks and the system window

Unless `show_heat_sink` is false, every dissipating component runs a pathway downwards which converges, with all the others, on a single ground symbol. Dissipation is flow to a common sink, not contact with a surface, so there is no horizontal floor line.

The dissipating types are `producer`, `consumer`, `storage`, `interaction`, `misc`, `gain`, `loop_limited` and `switch` — the symbols Odum's sheet draws with a heat sink. Sources and exchanges dissipate nothing, and `text`, `box` and `ground` are plain shapes with no `heat_sink` anchor.

A component's dissipation may instead be taken off one of its pathways, as Odum draws a storage's depreciation as a branch off its outflow: `heat` on the edge draws the branch from the middle of the pathway, marked at the junction by a small filled triangle, and the component then runs no heat pathway of its own.

The bundle ends in one large filled arrowhead resting on the ground's top bar, as in Odum's figures; `heat_sink_label` writes the total beside it.

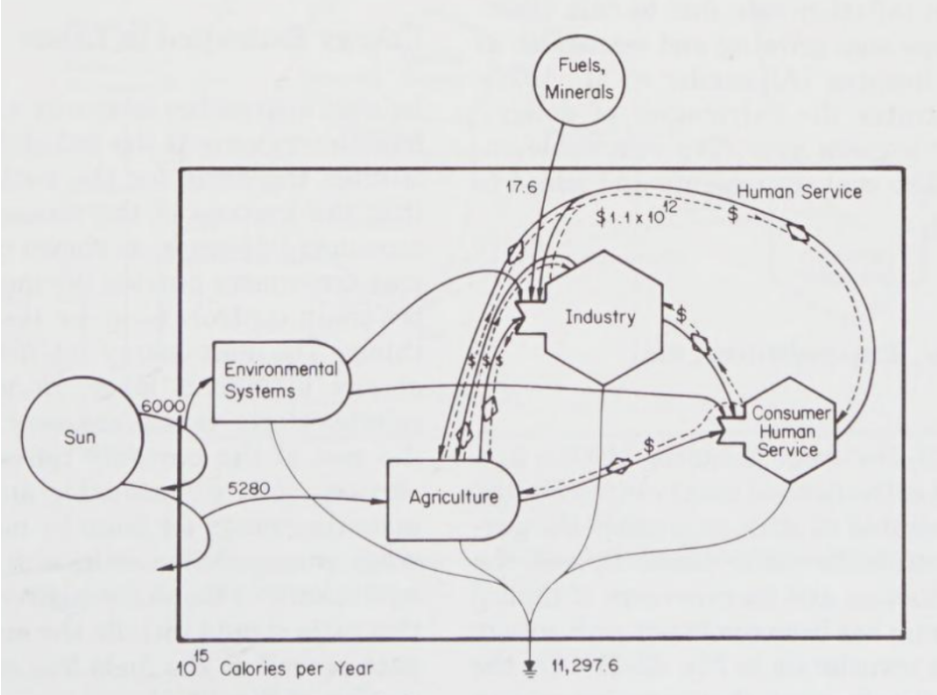
Where a system window is drawn, the dissipation pathways converge just inside it and a single line crosses the frame to reach the ground outside. Each pathway crossing separately would shred the boundary, and would misstate the accounting: what leaves the system is one aggregate flow of used energy, not several.

`system_boundary` draws an optional *system window*: the frame declaring what the model treats as inside the system, and therefore what its macro-level inputs and outputs are. Its `label` — usually the units of the diagram's quantities — is written under the frame's bottom-left corner, as Odum writes "10^{15} Calories per Year". The ground symbol is always placed *outside* that window — used energy leaves the system, so its terminator cannot sit within the frame. A `heat_sink_y` that would put the ground inside the window is overridden rather than honoured.

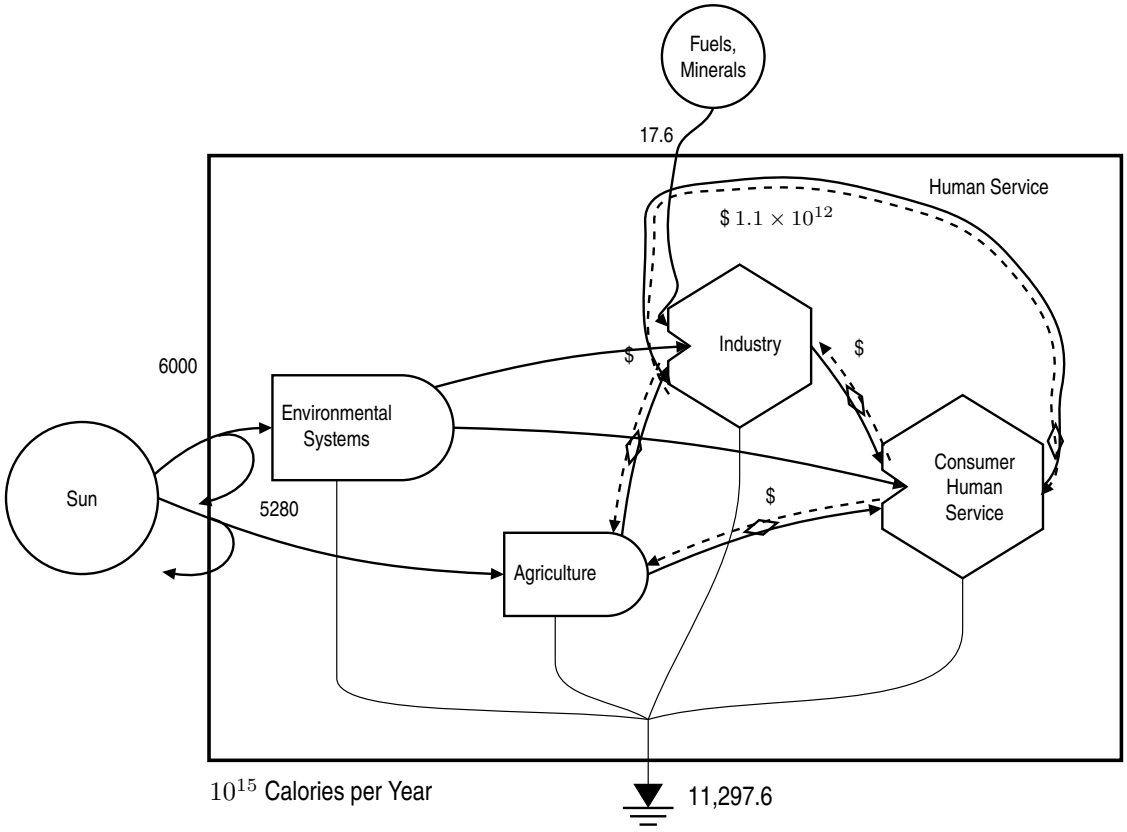
The pathways nest: the component furthest from the ground turns in lowest and runs outside the nearer ones all the way in, so none of them cross. That needs room under the components, and a frame given explicit coordinates that leaves too little is deepened to make it, with a warning giving the new `y_min`. Copy that value into the model to silence it.

13 A worked example

`examples/aggregated-economy.json` reproduces Odum's aggregated economy. It is traced: every position in it was measured from his figure, reproduced below beside it, so the composition is his rather than the layout engine's. It exercises a system window, notched consumers, four exchanges — one of them the human service fed back to industry over the top of the whole system — the Sun's unused-energy hooks, quantities on pathways, and the heat-sink total.



Odum's figure (`reference-images/aggregated-economy.png`)



`\renderEnergese{examples/aggregated-economy.json}`